



Edge Deployment of Federated Models

From training to real-world devices: Implementing lightweight, efficient, and secure AI at the edge

Why Edge Deployment?



Edge devices include:

Smartphones, IoT sensors, wearables, robots

Low Latency

Real-time inference without network delays

Offline Operation

No dependency on always-on cloud connectivity

Enhanced Privacy

Data never leaves the device

Cost Reduction

Less bandwidth and cloud compute requirements

Workflow of Edge FL Deployment



Train Global Model

Use Federated Learning to create initial model



Optimize & Compress

QAT, pruning, distillation to reduce size



Deploy to Edge

Distribute model to target devices



Local Processing

Inference → update → secure aggregation



Iterate Updates

Push global updates across all clients

The continuous cycle enables models to improve while preserving privacy and efficiency.

Challenges in Edge Deployment



- 1 Resource Limitations**
Constrained memory (often <100MB), limited battery life, reduced compute capability
- 2 Connectivity Issues**
Intermittent network access, varying bandwidth, high latency
- 3 Device Heterogeneity**
Wide range of CPUs, NPUs, GPUs, microcontrollers with different capabilities
- 4 Security Vulnerabilities**
Risk of adversarial updates, model theft, compromised devices

Model Optimization for Edge

Converting research models into deployment-ready assets

Quantization (INT8)

- Reduce precision from FP32 to INT8
- 4x smaller model size
- 2-4x inference speedup
- Minimal accuracy loss with QAT

Pruning

- Remove redundant weights/neurons
- Structured sparsity for hardware acceleration
- 50-90% parameter reduction possible
- Requires retraining for best results

Knowledge Distillation

- Train compact "student" models
- Learn from larger "teacher" models
- Preserve inference quality
- Often combined with quantization

On-device compilers like CoreML, TensorRT, and TFLite provide additional hardware-specific optimizations.



Tools for Edge Deployment



TensorFlow Lite

Optimized for Android and IoT devices. Supports quantization, hardware acceleration, and model optimization.



PyTorch Mobile

End-to-end workflow for deploying PyTorch models to mobile devices with TorchScript for optimization.



Core ML

Apple's framework for iOS devices, leveraging the Neural Engine for hardware acceleration.



ONNX Runtime

Cross-platform inference engine supporting multiple frameworks and hardware accelerators.

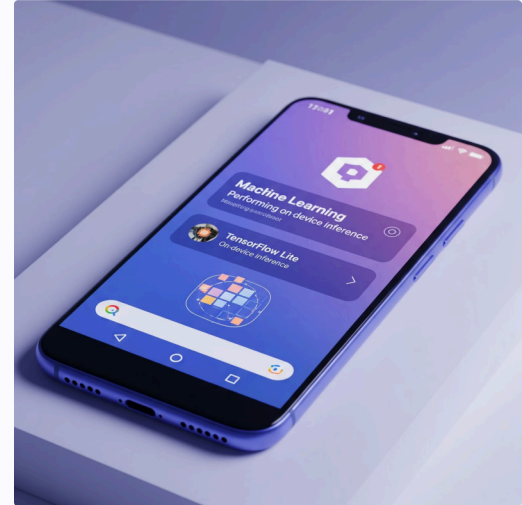
Example: Deploying to Android with TFLite

```
import tensorflow as tf

# Convert model to TFLite
converter = tf.lite.TFLiteConverter.from_keras_model(model)
converter.optimizations = [tf.lite.Optimize.DEFAULT]
tflite_model = converter.convert()

# Save
with open("model.tflite", "wb") as f:
    f.write(tflite_model)
```

The optimized .tflite file can run efficiently on mobile CPU/NPU



Key benefits:

- Automatic quantization with DEFAULT optimization
- Reduced model size (often 4x smaller)
- Hardware acceleration on compatible devices
- Simple integration with Android Studio

Example: PyTorch Mobile

```
import torch

# Convert to TorchScript
traced_model = torch.jit.trace(model, example_input)
torch.jit.save(traced_model, "model.pt")

# Load on mobile app (Android/iOS)
# Use PyTorch Mobile runtime for inference
```

TorchScript enables lightweight mobile inference across platforms.



Implementation Steps:

1. Trace or script your model
2. Apply optimizations (quantization, pruning)
3. Export to mobile-ready format
4. Integrate with mobile app using PyTorch Mobile runtime
5. Implement on-device inference pipeline

Edge Hardware Acceleration



Smartphone NPUs

Apple Neural Engine, Qualcomm Hexagon DSP, Samsung Exynos NPU provide 5-15x speedup over CPU



TinyML Devices

ARM Cortex-M with optimized libraries run ML on microcontrollers with <1MB RAM



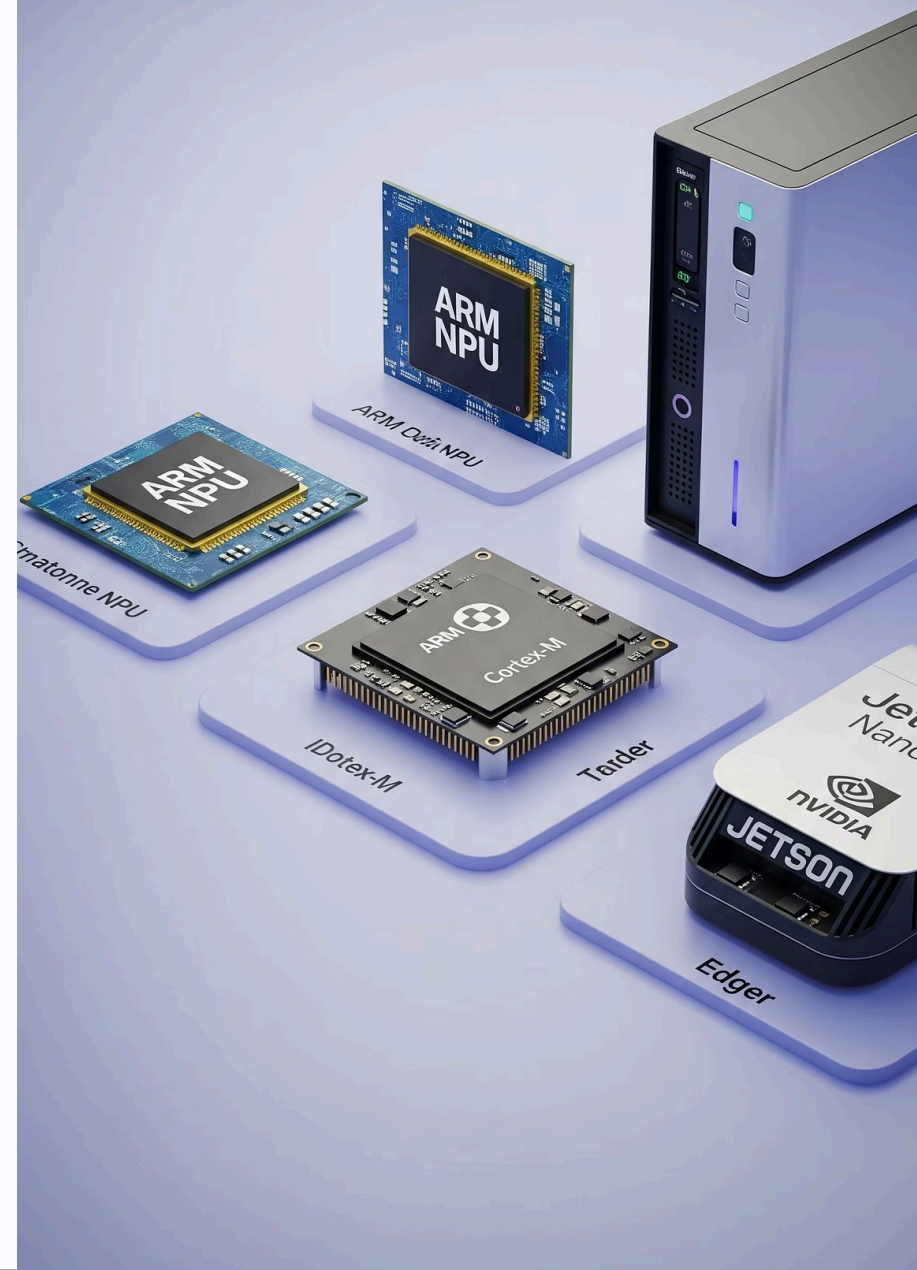
Edge GPUs

NVIDIA Jetson Nano/Xavier enable desktop-class inference in edge form factors



Cloud-Edge Hybrid

Split execution between device and cloud for optimal performance/efficiency



Security in Edge FL Deployment



Secure Aggregation

Cryptographic protocols ensure individual updates remain private while enabling global model improvement

Differential Privacy

Add calibrated noise to model updates to prevent reconstruction of training data

Trusted Execution Environments

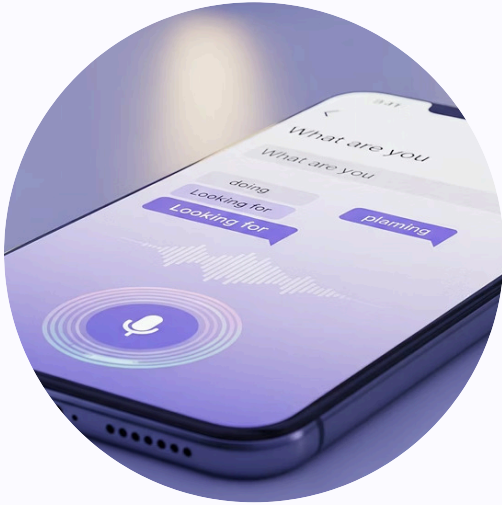
Isolated secure processing areas (ARM TrustZone, Intel SGX) for sensitive inference

Device Attestation

Verify device identity and integrity before accepting model updates

Security requirements increase computational overhead but are essential for production deployments.

Real-World Applications



Smartphones

Predictive text, speech recognition, image processing, and smart assistants leverage on-device ML



Healthcare Wearables

Heart rate anomaly detection, sleep analysis, and activity monitoring with privacy protection



Autonomous Vehicles

Edge vision processing, sensor fusion, and real-time decision making with low latency requirements

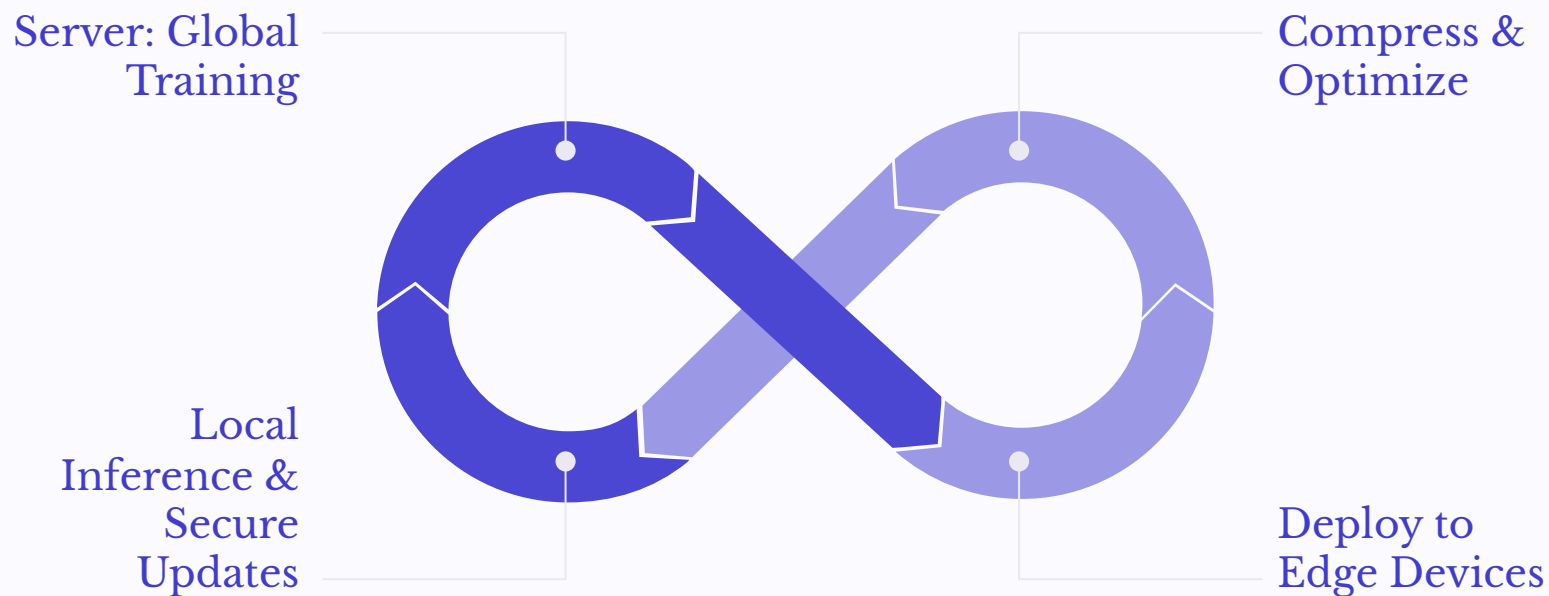


Retail IoT

Smart cameras for inventory tracking, loss prevention, and customer behavior analysis

Deployment Pipeline

Closing the FL loop with edge deployment



The circular workflow enables continuous improvement while maintaining privacy and reducing bandwidth requirements.

Tradeoffs in Edge Deployment

Model Size vs. Accuracy

Smaller models improve performance but may sacrifice prediction quality

Security vs. Efficiency

Heavy encryption and privacy protections increase latency and battery consumption

Offline Support vs. Update Frequency

More autonomous operation may mean less frequent model improvements

Development Complexity vs. Deployment Flexibility

Supporting multiple hardware targets increases engineering overhead



Finding the Right Balance

Successful edge deployments require balancing **accuracy**, **efficiency**, **security**, and **usability** based on specific use case requirements.

Summary

Key Takeaways

- Edge deployment transforms FL research into practical applications
- Optimization techniques are essential: quantization, pruning, distillation
- Framework tools include TFLite, PyTorch Mobile, Core ML, ONNX Runtime
- Security must be built into the deployment pipeline
- Balance tradeoffs based on specific application needs



- ❏ Edge deployment of federated models is critical for building **scalable, privacy-first AI ecosystems** that can operate efficiently on billions of devices worldwide.